

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
In [23]: #Load the dataset
df= pd.read_csv(r"C:\Users\VYSHNAVI\AppData\Local\Temp\c0d2a27c-83de-42fa-b073-b82148f0238d_a

print(df.head(),"\n")
print(df.shape,"\n")
print(df.info())
```

	StudentID	CGPA	Internships	Projects	Workshops/Certifications	\
0	1	7.5	1	1		1
1	2	8.9	0	3		2
2	3	7.3	1	2		2
3	4	7.5	1	1		2
4	5	8.3	1	2		2

	AptitudeTestScore	SoftSkillsRating	ExtracurricularActivities	\
0	65	4.4	No	
1	90	4.0	Yes	
2	82	4.8	Yes	
3	85	4.4	Yes	
4	86	4.5	Yes	

	PlacementTraining	SSC_Marks	HSC_Marks	PlacementStatus
0	No	61	79	NotPlaced
1	Yes	78	82	Placed
2	No	79	80	NotPlaced
3	Yes	81	80	Placed
4	Yes	74	88	Placed

(10000, 12)

<class 'pandas.DataFrame'>

RangeIndex: 10000 entries, 0 to 9999

Data columns (total 12 columns):

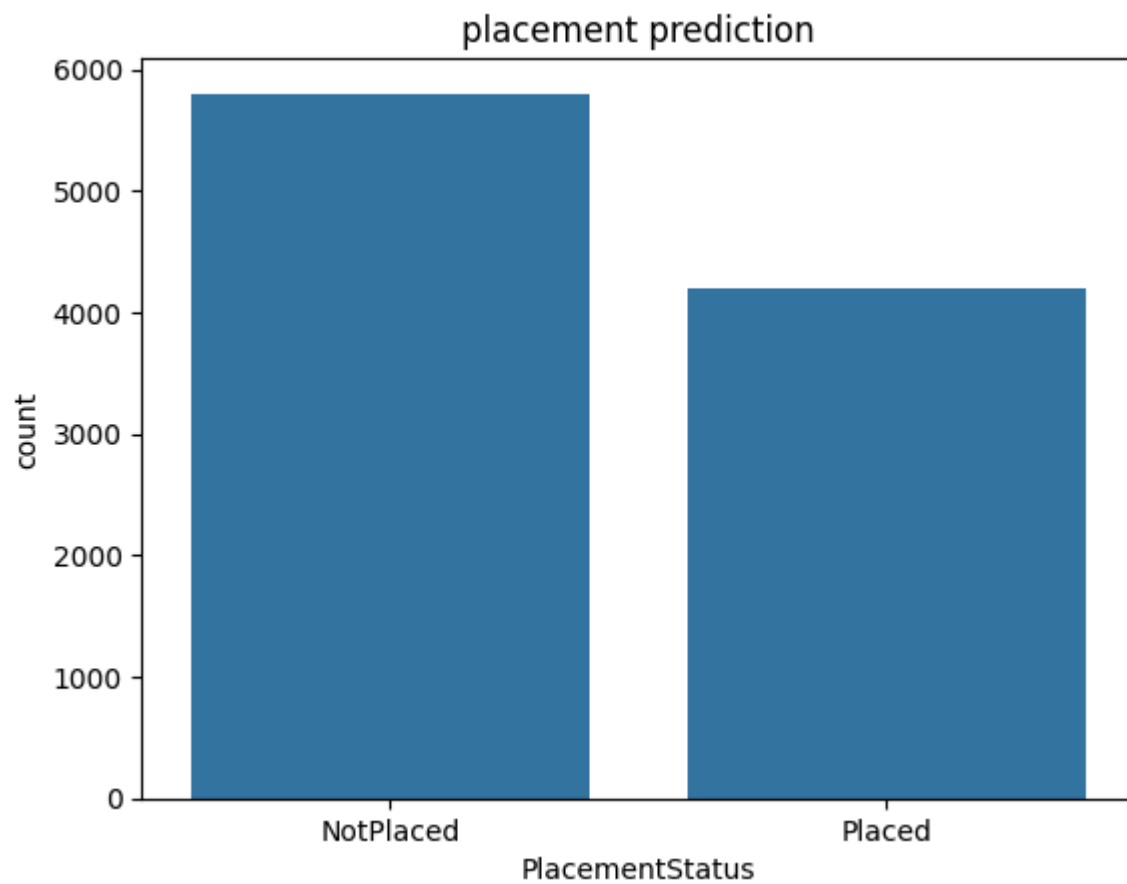
#	Column	Non-Null Count	Dtype
0	StudentID	10000 non-null	int64
1	CGPA	10000 non-null	float64
2	Internships	10000 non-null	int64
3	Projects	10000 non-null	int64
4	Workshops/Certifications	10000 non-null	int64
5	AptitudeTestScore	10000 non-null	int64
6	SoftSkillsRating	10000 non-null	float64
7	ExtracurricularActivities	10000 non-null	str
8	PlacementTraining	10000 non-null	str
9	SSC_Marks	10000 non-null	int64
10	HSC_Marks	10000 non-null	int64
11	PlacementStatus	10000 non-null	str

dtypes: float64(2), int64(7), str(3)

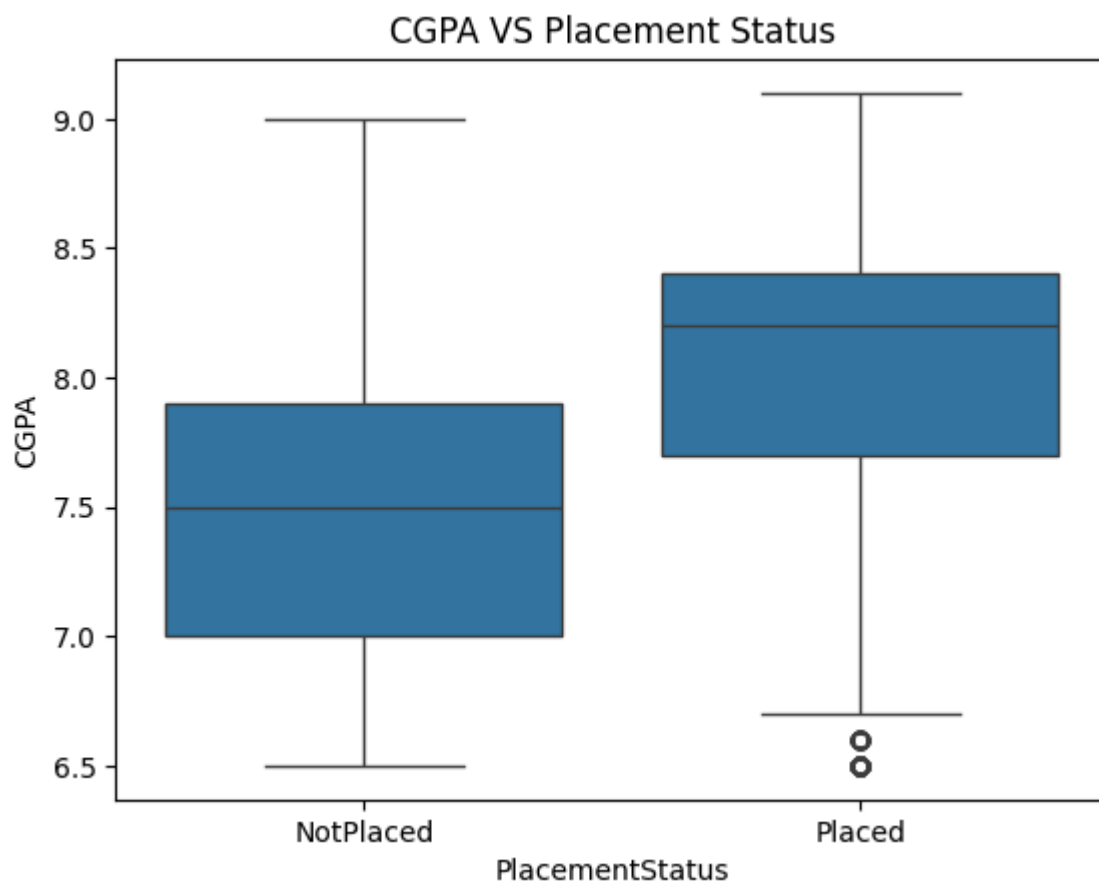
memory usage: 937.6 KB

None

```
In [ ]: #placed distribution
sns.countplot(x='PlacementStatus',data=df)
plt.title("placement prediction")
plt.show()
```



```
In [5]: #cgpa vs placementstatus
sns.boxplot(x='PlacementStatus',y='CGPA',data=df)
plt.title("CGPA VS Placement Status")
plt.show()
```



```
In [5]: #missing values
print(df.isnull().sum())
```

```

StudentID      0
CGPA           0
Internships    0
Projects       0
Workshops/Certifications 0
AptitudeTestScore 0
SoftSkillsRating 0
ExtracurricularActivities 0
PlacementTraining 0
SSC_Marks      0
HSC_Marks      0
PlacementStatus 0
dtype: int64

```

```

In [6]: #remove duplicates
df.drop_duplicates()

```

```

Out[6]:

```

	StudentID	CGPA	Internships	Projects	Workshops/Certifications	AptitudeTestScore	SoftSkillsR
0	1	7.5	1	1	1	65	
1	2	8.9	0	3	2	90	
2	3	7.3	1	2	2	82	
3	4	7.5	1	1	2	85	
4	5	8.3	1	2	2	86	
...	...	...	...	...	...	...	...
9995	9996	7.5	1	1	2	72	
9996	9997	7.4	0	1	0	90	
9997	9998	8.4	1	3	0	70	
9998	9999	8.9	0	3	2	87	
9999	10000	8.4	0	1	1	66	

10000 rows × 12 columns



```

In [7]: #removing unnecessary columns
df.drop(
    ['SSC_Marks', 'HSC_Marks'],
    axis=1,
    inplace=True
)

```

```

In [24]: df['PlacementTraining']=df['PlacementTraining'].map({'Yes': 1, 'No': 0 })
df['PlacementStatus'] = df['PlacementStatus'].map({ 'Placed': 1, 'NotPlaced': 0 })
df['ExtracurricularActivities']=df['ExtracurricularActivities'].map({'Yes': 1, 'No': 0 })

```

```

In [9]: print(df['PlacementTraining'].dtype)
print(df['PlacementStatus'].dtype)
print(df['ExtracurricularActivities'].dtype)

```

```

int64
int64
int64

```

```

In [15]: #separate features and target
X = df.drop('PlacementStatus',axis=1)
y = df['PlacementStatus']

```

```
In [16]: #split dataset
X_train, X_test, y_train , y_test = train_test_split(
    X, y,
    test_size=0.4,
    random_state=53
)

model = RandomForestClassifier(
    n_estimators=300,
    random_state=53
)

model.fit(X_train, y_train)
print("model trained successfully")
```

model trained successfully

```
In [19]: #model prediction
y_pred = model.predict(X_test)
print(y_pred)
print(y_test)
```

```
[0 0 1 ... 0 0 0]
2836    0
7507    0
1615    1
4284    1
7875    0
..
5402    1
94      0
1196    1
3415    1
995     0
Name: PlacementStatus, Length: 4000, dtype: int64
```

```
In [20]: #accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.7925

```
In [22]: #Alertnative model
from sklearn.linear_model import LogisticRegression

model1=LogisticRegression(max_iter=80)
model1.fit(X_train,y_train)
y_pred1=model1.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred1))
```

Accuracy: 0.76425

c:\Users\VYSHNAVI\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 80 iteration(s) (status=1):

STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=80).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

```
In [26]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

#Load the dataset
df= pd.read_csv(r"C:\Users\VYSHNAVI\AppData\Local\Temp\c0d2a27c-83de-42fa-b073-b82148f0238d_a

print(df.head(),"\n")
print(df.shape,"\n")
print(df.info())

#placed distribution
sns.countplot(x='PlacementStatus',data=df)
plt.title("placement prediction")
plt.show()

#cgpa vs placementstatus
sns.boxplot(x='PlacementStatus',y='CGPA',data=df)
plt.title("CGPA VS Placement Status")
plt.show()

#missing values
print(df.isnull().sum())

#remove duplicates
df.drop_duplicates()

#removing unnecessary columns
df.drop(
    ['SSC_Marks','HSC_Marks'],
    axis=1,
    inplace=True
)

df['PlacementTraining']=df['PlacementTraining'].map({'Yes': 1, 'No': 0 })
df['PlacementStatus'] = df['PlacementStatus'].map({'Placed': 1, 'NotPlaced': 0 })
df['ExtracurricularActivities']=df['ExtracurricularActivities'].map({'Yes': 1, 'No': 0 })

print(df['PlacementTraining'].dtype)
print(df['PlacementStatus'].dtype)
print(df['ExtracurricularActivities'].dtype)

#separate features and target
X = df.drop('PlacementStatus',axis=1)
y = df['PlacementStatus']

#split dataset
X_train, X_test, y_train , y_test = train_test_split(
    X, y,
    test_size=0.4,
    random_state=53
)

model = RandomForestClassifier(
    n_estimators=300,
    random_state=53
)

model.fit(X_train, y_train)
print("model trained successfully")

#model prediction
y_pred = model.predict(X_test)
print(y_pred)
print(y_test)

```

```
#accuracy
print("Accuracy:", accuracy_score(y_test,y_pred))

#Alertnative model
from sklearn.linear_model import LogisticRegression

model1=LogisticRegression(max_iter=80)
model1.fit(X_train,y_train)
y_pred1=model1.predict(X_test)
print("Accuracy:", accuracy_score(y_test,y_pred1))
```

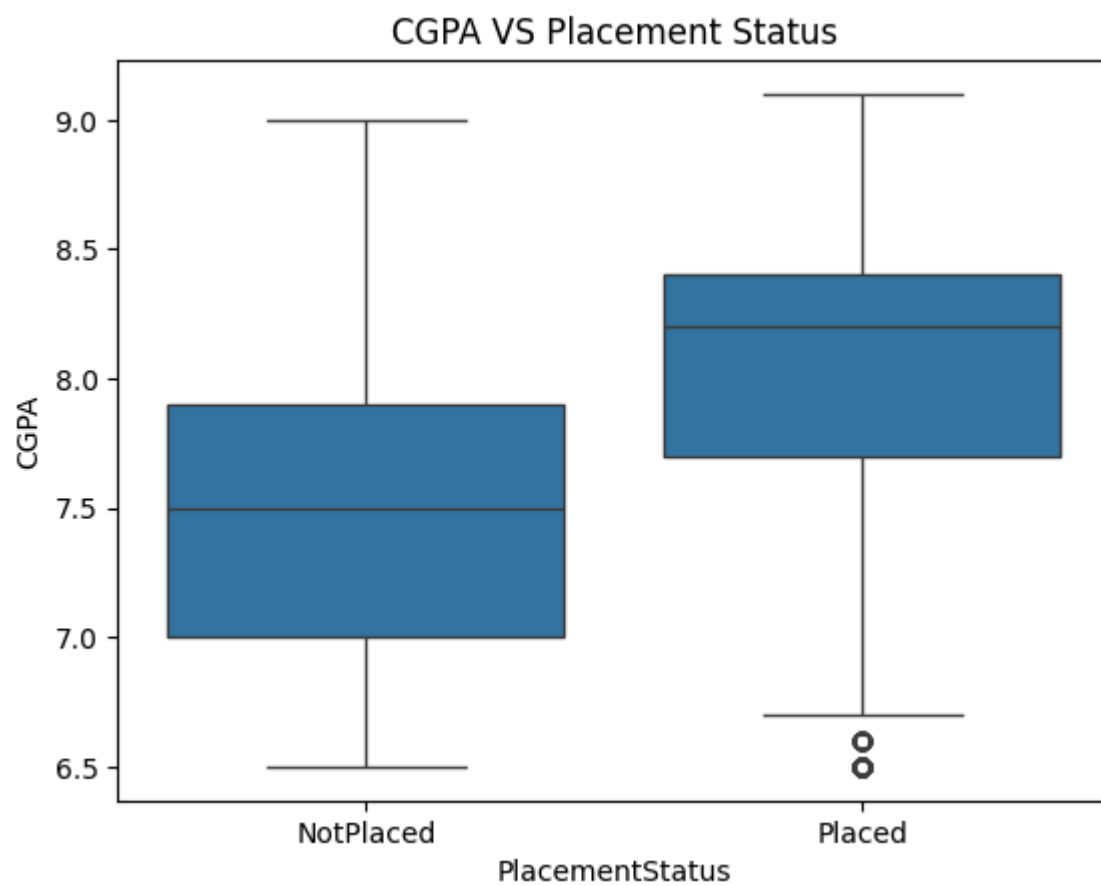
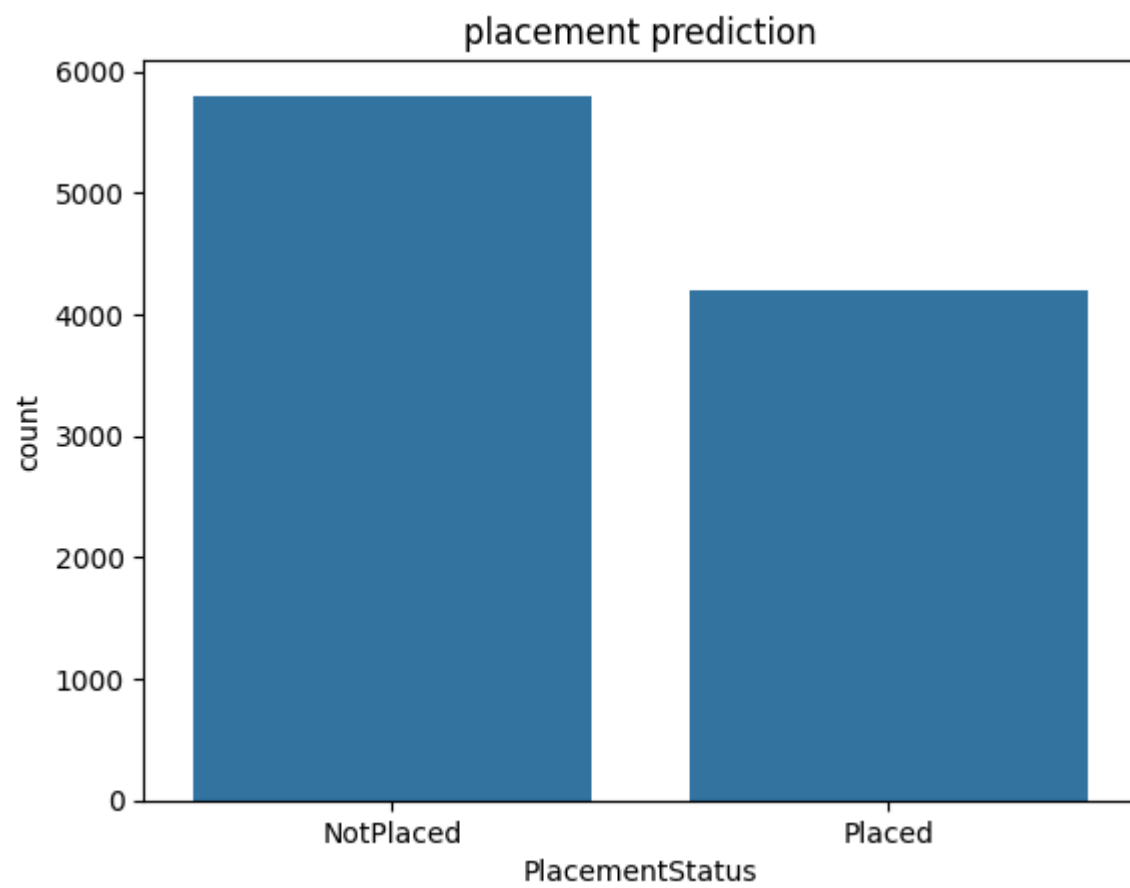
	StudentID	CGPA	Internships	Projects	Workshops/Certifications	\
0	1	7.5	1	1		1
1	2	8.9	0	3		2
2	3	7.3	1	2		2
3	4	7.5	1	1		2
4	5	8.3	1	2		2

	AptitudeTestScore	SoftSkillsRating	ExtracurricularActivities	\
0	65	4.4	No	
1	90	4.0	Yes	
2	82	4.8	Yes	
3	85	4.4	Yes	
4	86	4.5	Yes	

	PlacementTraining	SSC_Marks	HSC_Marks	PlacementStatus
0	No	61	79	NotPlaced
1	Yes	78	82	Placed
2	No	79	80	NotPlaced
3	Yes	81	80	Placed
4	Yes	74	88	Placed

(10000, 12)

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   StudentID                            10000 non-null  int64
1   CGPA                                  10000 non-null  float64
2   Internships                           10000 non-null  int64
3   Projects                              10000 non-null  int64
4   Workshops/Certifications              10000 non-null  int64
5   AptitudeTestScore                    10000 non-null  int64
6   SoftSkillsRating                     10000 non-null  float64
7   ExtracurricularActivities             10000 non-null  str
8   PlacementTraining                    10000 non-null  str
9   SSC_Marks                            10000 non-null  int64
10  HSC_Marks                             10000 non-null  int64
11  PlacementStatus                       10000 non-null  str
dtypes: float64(2), int64(7), str(3)
memory usage: 937.6 KB
None
```



```
StudentID      0
CGPA           0
Internships    0
Projects       0
Workshops/Certifications 0
AptitudeTestScore 0
SoftSkillsRating 0
ExtracurricularActivities 0
PlacementTraining 0
SSC_Marks      0
HSC_Marks      0
PlacementStatus 0
```

```
dtype: int64
```

```
int64
```

```
int64
```

```
int64
```

```
model trained successfully
```

```
[0 0 1 ... 0 0 0]
```

```
2836    0
```

```
7507    0
```

```
1615    1
```

```
4284    1
```

```
7875    0
```

```
..
```

```
5402    1
```

```
94      0
```

```
1196    1
```

```
3415    1
```

```
995     0
```

```
Name: PlacementStatus, Length: 4000, dtype: int64
```

```
Accuracy: 0.7925
```

```
Accuracy: 0.76425
```

```
c:\Users\VYSHNAVI\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\linear_model\_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 80 iteration(s) (status=1):
```

```
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

```
Increase the number of iterations to improve the convergence (max_iter=80).
```

```
You might also want to scale the data as shown in:
```

```
https://scikit-learn.org/stable/modules/preprocessing.html
```

```
Please also refer to the documentation for alternative solver options:
```

```
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
```

```
n_iter_i = _check_optimize_result(
```